Table des matières

Cours 02 — Fonctionnalités AI/ML et développement 1
Objectifs 1
Théorie 1
QCM — style examen (Cours 02) 6
Points clés à retenir 10

Cours 02 — Fonctionnalités AI/ML et développement

Module 2/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Comment lire ce module — Il boucle le domaine 1 (le plus lourd, 31 %) : le Cours 01 a pris l'architecture et les objets ; celui-ci prend le volet développement et IA, tout ce qu'on construit et exécute *dans* Snowflake. Le niveau attendu à l'examen est « reconnaître » : on ne vous apprend pas à *coder* Snowpark ou Cortex, on vous apprend à savoir ce que fait chaque outil, à quoi il sert et quand le citer. Gardez ce réflexe : *reconnaître, pas réciter du code.*

Objectifs

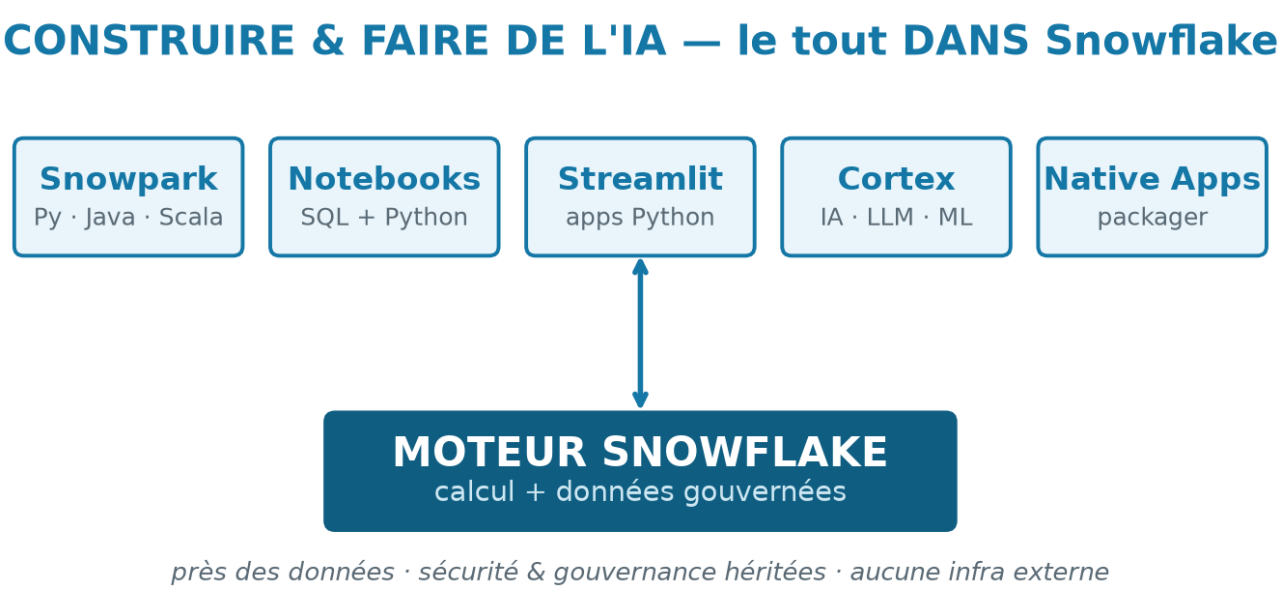
À la fin de ce module, vous saurez :

- expliquer l'idée commune : rapprocher le code (et l'IA) des données, sans infrastructure externe ;
- dire ce qu'est Snowpark et pourquoi un DataFrame Snowpark est *paresseux* et poussé en SQL ;
- situer les variantes : entrepôts Snowpark-optimized, Container Services, Snowpark ML, Snowpark Connect for Spark ;
- distinguer une fonction définie par l'utilisateur (UDF), sa variante qui rend une table (UDTF) et une procédure stockée, et connaître leurs langages ;
- reconnaître Snowflake Notebooks et Streamlit in Snowflake ;
- reconnaître Cortex : les fonctions LLM et ML en SQL, plus Search, Analyst, Document AI ;
- situer le Native App Framework (packager une app ; la distribution, c'est le Cours 06).

Théorie

1. Développer *dans* Snowflake — l'idée

© 2026 iAlexMG. Tous droits réservés.



Les cinq portes du « construire dans Snowflake », Snowpark, Notebooks, Streamlit, Cortex et Native Apps, toutes posées sur le même moteur.

Au Cours 01, on a séparé accéder à Snowflake et construire dessus. Voici le « construire ». Le principe : au lieu de sortir les données vers un serveur Python, un cluster Spark ou une API d'IA, on amène le calcul et l'IA à la donnée. Trois gains, toujours les mêmes :

- Proximité — le code tourne sur les entrepôts Snowflake, à côté du stockage : pas de copie à exporter, moins de latence.
- Zéro infrastructure — rien à installer ni administrer côté serveurs ; Snowflake gère le calcul.
- Gouvernance héritée — mêmes rôles, masquage et politiques que le reste : votre code Python ou votre appel d'IA respecte automatiquement les droits en place.

Deux grandes familles à garder en tête : Snowpark (vous écrivez du code) et Cortex (vous appelez de l'IA prête à l'emploi). Autour gravitent Notebooks (là où vous écrivez), Streamlit (des applis) et Native Apps (empaqueter pour d'autres).

2. Snowpark — exécuter Python / Java / Scala dans Snowflake

Labo : *`Labo_02_SNOW.md`, étape 1.*

© 2026 iAlexMG. Tous droits réservés.

Snowpark est un ensemble de bibliothèques et d'exécutions qui font tourner du code Python, Java ou Scala directement sur le moteur Snowflake. Son cœur, c'est l'API DataFrame.

- Un DataFrame, c'est un tableau de données manipulé par du code (filtrer, joindre, agréger…), l'équivalent programmatique d'une requête.
- Le point clé pour l'examen : le DataFrame Snowpark est paresseux (*lazy*). Vos opérations ne s'exécutent pas tout de suite : elles construisent un plan. Rien ne part tant que vous ne déclenchez pas une action (afficher, compter, écrire). À ce moment, Snowpark traduit le plan en SQL et le fait exécuter par l'entrepôt, et les données ne quittent pas Snowflake.
- À comparer avec pandas (la bibliothèque Python classique) : pandas travaille en mémoire sur votre machine, donc il faut rapatrier les données. Snowpark pousse le calcul vers Snowflake (*pushdown*) : ça passe à l'échelle sans rien télécharger.

```
# Illustration (pas à mémoriser) : rien ne s'exécute avant .show()
df = session.table("VENTES").filter(col("PAYS") == "CA").group_by("MOIS").count()
df.show()   # <- ICI Snowpark envoie le SQL à l'entrepôt
```

Quelques variantes à reconnaître :

- Snowpark for Python — la déclinaison la plus courante : API DataFrame et moyen d'écrire des UDF, UDTF et procédures en Python (section 3).
- Entrepôts *Snowpark-optimized* — des entrepôts avec plus de mémoire par nœud, pour les charges gourmandes (entraînement de modèles, gros objets en mémoire). *(Vu au Cours 01, section 5.)*
- Snowpark Container Services — exécuter des conteneurs (images Docker), y compris avec GPU, gérés par Snowflake. Pour des services/applications qui débordent du simple SQL. *(Reconnaître.)*
- Snowpark ML / Snowflake ML — une bibliothèque Python pour le feature engineering et l'entraînement/inférence de modèles, sans sortir de Snowflake. *(Reconnaître.)*
- Snowpark Connect for Spark — permet de faire tourner du code Spark (l'API DataFrame de Spark, via le protocole *Spark Connect*) sur le moteur Snowflake, sans gérer de cluster Spark séparé. L'idée : réutiliser des charges Spark existantes en les faisant exécuter par Snowflake. *(C'est la question fréquente « Snowpark vs Spark » : Snowpark = l'API native de Snowflake ; Snowpark Connect = un pont pour le code Spark.)*

3. Le code réutilisable : UDF, UDTF et procédures stockées

Labo : `Labo_02_SNOW.md`, étape 1.

Snowpark (et le SQL) servent souvent à fabriquer des objets de code réutilisables. Trois types, à ne pas confondre :

© 2026 iAlexMG. Tous droits réservés.

Objet	Renvoie…	Sert à…	On l'appelle…
UDF (fonction)	une valeur (par ligne)	calculer quelque chose dans une requête	dans un `SELECT`
UDTF (fonction *de table*)	des lignes (une table)	produire plusieurs lignes par entrée	dans un `FROM … TABLE(...)`
Procédure stockée	(une valeur, souvent un statut)	orchestrer : enchaîner du SQL, boucles, conditions	avec `CALL`

- Langages : une UDF/UDTF peut être écrite en SQL, JavaScript, Python, Java ou Scala ; une procédure en SQL (Snowflake Scripting), JavaScript, Python, Java ou Scala. *(Retenez la richesse des langages, pas la liste par cœur.)*
- UDF vs procédure, la vraie différence : une UDF *calcule une valeur* qu'on utilise dans une requête (elle ne fait pas d'« actions ») ; une procédure *fait des choses* (exécuter du SQL, gérer des étapes) et se déclenche par `CALL`.
- Fonctions externes — un cas à part : elles appellent du code hébergé en dehors de Snowflake (via une passerelle d'API). *(Reconnaître.)*

4. Snowflake Notebooks

Labo : `Labo_02_SNOW.md`, étape 4.

Un Notebook est un document fait de cellules qu'on exécute une à une, mêlant SQL, Python et Markdown (du texte). Snowflake en offre directement dans Snowsight : les cellules tournent sur du calcul Snowflake (un entrepôt, ou une exécution conteneurisée), et le Python peut utiliser Snowpark. C'est l'atelier idéal pour explorer des données, prototyper du ML et visualiser, le tout dans la plateforme, sans installer d'environnement local.

5. Streamlit in Snowflake

Labo : `Labo_02_SNOW.md`, étape 4.

Streamlit est une bibliothèque Python pour bâtir des applications de données (tableaux de bord interactifs, formulaires) sans faire de développement web. Streamlit *in* Snowflake permet de créer et héberger ces applis à l'intérieur de Snowflake : elles s'exécutent sur son calcul, héritent de la sécurité et de la gouvernance, et se partagent avec d'autres utilisateurs du compte, sans aucun serveur externe à gérer.

6. Cortex — l'IA intégrée

Labo : `Labo_02_SNOW.md`, étapes 2 et 3.

Snowflake Cortex est la couche d'intelligence artificielle de la plateforme : on l'appelle en SQL (et en Python), sans monter d'infrastructure d'IA. Deux familles à distinguer nettement.

© 2026 iAlexMG. Tous droits réservés.

a) Les fonctions d'IA — appeler un grand modèle de langage (*LLM*, l'IA générative type ChatGPT) depuis une simple requête. Elles sont serverless : Snowflake fournit le calcul et le facture à l'usage, sans entrepôt à créer ni à dimensionner (ici, la facture se compte en jetons consommés). Elles portent toutes un nom en `AI_` :

```
SELECT AI_SUMMARIZE(avis)              AS resume,
       AI_SENTIMENT(avis)              AS tonalite,
       AI_TRANSLATE(avis, 'en', 'fr')  AS traduction
FROM   avis_clients;
```

Au menu, les plus citées : `AI_COMPLETE` (réponse libre d'un LLM, la porte d'entrée de la famille), `AI_SUMMARIZE`, `AI_TRANSLATE`, `AI_SENTIMENT`, `AI_CLASSIFY`, `AI_FILTER` (vrai ou faux sur un texte, donc utilisable dans un `WHERE`), `AI_EXTRACT`, `AI_EMBED` (transformer un texte en vecteur, pour la recherche) et `AI_PARSE_DOCUMENT`. L'offre bouge vite : la liste qui fait foi est celle de la documentation.

Piège — deux générations de noms, et l'examen peut citer l'ancienne. Avant les `AI_*`, ces mêmes services s'appelaient `SNOWFLAKE.CORTEX.COMPLETE`, `SUMMARIZE`, `TRANSLATE`, `SENTIMENT`. Ces fonctions existent toujours et s'exécutent, mais la documentation range désormais leurs pages sous « backward compatibility » et annonce la dépréciation des plus anciennes pour la fin de 2026, en renvoyant vers les `AI_*`, « the canonical surface going forward ». Reconnaissez les deux écritures ; écrivez la neuve.

Une différence qui ne se devine pas : `SNOWFLAKE.CORTEX.SENTIMENT` rend un nombre de −1 à +1, alors que `AI_SENTIMENT` rend un objet, avec le sentiment d'ensemble et, si vous les demandez, celui de chaque aspect (le prix, le service, l'attente). Ce n'est pas le même résultat, donc pas un simple changement de nom.

Note d'accès : il faut deux choses, et l'examen aime cette paire. Le privilège de compte `USE AI FUNCTIONS`, accordé au rôle `PUBLIC` par défaut, donc à tout le monde tant qu'un administrateur ne le retire pas. Et un des deux rôles de base de données, `SNOWFLAKE.CORTEX_USER` ou `AI_FUNCTIONS_USER`. S'ajoute la disponibilité selon la région du compte.

b) Les fonctions ML — du machine learning classique sur des données tabulaires, sans écrire de code de data science. Vous donnez des données, la fonction fait le modèle :

- `FORECAST` — prévision de série temporelle (ventes des prochaines semaines…).
- `ANOMALY_DETECTION` — repérer les valeurs aberrantes.
- `CLASSIFICATION` — classer des lignes en catégories.
- `TOP_INSIGHTS` — trouver ce qui explique une variation.

Ces quatre-là gardent leurs noms historiques : la vague des `AI_*` n'a touché que les fonctions de texte et d'image.

Et trois briques de plus haut niveau, à reconnaître :

© 2026 iAlexMG. Tous droits réservés.

- Cortex Search — un service de recherche (mots-clés + sémantique) sur vos textes ; la base des systèmes de RAG (*Retrieval-Augmented Generation* : donner à un LLM les bons documents avant qu'il réponde).
- Cortex Analyst — poser une question en langage naturel et obtenir la réponse ; il traduit en SQL sur un *modèle sémantique* que vous décrivez.
- Document AI — extraire des informations structurées de documents (factures, contrats…).

7. Native Apps — empaqueter pour d'autres

Le Snowflake Native App Framework sert à empaqueter de la logique (UDF, procédures, Streamlit, modèles…) et des données en une application que d'autres comptes installent et exécutent chez eux, et les données ne quittent pas le compte du consommateur. C'est la façon « produit » de distribuer ce qu'on a bâti aux sections précédentes. Ici on le reconnaît ; sa distribution (Marketplace, *listings*, partage) est le Cours 06.

QCM — style examen (Cours 02)

Comment répondre — Faites les neuf questions avant d'ouvrir le corrigé, qui commence à la page suivante. Une question à choix multiple dit combien de réponses cocher, comme à l'examen. Comptez une minute par question.

1. Votre équipe traite 200 millions de lignes avec pandas, sur un poste, en téléchargeant la table depuis Snowflake. On vous demande de passer à Snowpark. Qu'est-ce qui change ?
    a. Rien au fond : Snowpark télécharge les mêmes données, avec un protocole plus rapide.
    b. Le code devient du SQL que vous écrivez à la main, Snowpark n'étant qu'un traducteur.
    c. Les opérations bâtissent un plan, traduit en SQL et exécuté par l'entrepôt : les données ne sortent pas de Snowflake.
    d. Le calcul part sur un cluster Spark géré par Snowflake, qui remplace votre poste.
2. Votre script Snowpark enchaîne dix filtres et jointures, s'exécute en une seconde, et rien n'apparaît côté entrepôt. Que se passe-t-il ?
    a. Le DataFrame est paresseux : le plan ne part qu'à l'action (`show`, `count`, une écriture).
    b. L'entrepôt est suspendu, et Snowpark ne sait pas le réveiller seul.
    c. Il manque un `COMMIT`, chaque opération Snowpark ouvrant une transaction.
    d. Les filtres sont invalides, et Snowpark garde les erreurs pour la fin du script.
3. Deux besoins tombent le même jour : normaliser un code postal au fil d'un `SELECT`, sur des millions de lignes, et enchaîner chaque nuit cinq instructions SQL avec une condition entre elles.
    a. Une procédure stockée pour les deux, appelée par `CALL`.

© 2026 iAlexMG. Tous droits réservés.

b. Une UDF pour les deux, appelée dans le `SELECT`.

c. Une UDTF pour le code postal, une UDF pour l'enchaînement de nuit.

d. Une UDF pour le code postal, dans le `SELECT` ; une procédure pour l'enchaînement, lancée par `CALL`.

4. Une colonne porte la liste des articles d'une commande. Vous voulez une fonction qui rende une ligne par article, utilisable directement dans une requête.

   a. Une UDF : elle rend plusieurs lignes dès que son type de retour est une table.

   b. Une UDTF, appelée dans un `FROM … TABLE(...)`.

   c. Une procédure stockée, qui boucle sur les articles et les insère.

   d. Une vue matérialisée posée sur la colonne des articles.

5. *(choix multiple, sélectionnez-en DEUX)* Sans écrire de code de science des données, vous devez résumer 50 000 avis clients et prévoir les ventes des huit prochaines semaines. Deux appels font le travail.

   a. `AI_COMPLETE`, en lui décrivant la série des ventes en toutes lettres.

   b. La fonction d'IA `AI_SUMMARIZE`, en SQL, sur la colonne des avis.

   c. La fonction ML `FORECAST` de Cortex, sur l'historique des ventes.

   d. Un entrepôt Snowpark-optimized, qui porte les modèles de résumé et de prévision.

6. Une analyste veut livrer à son équipe un écran avec deux filtres et un graphique, hébergé dans Snowflake, sans serveur à monter ni développement web.

   a. Un Snowflake Notebook partagé, que chacun exécutera cellule par cellule.

   b. Un tableau de bord Snowsight, seul objet interactif de la plateforme.

   c. Une Native App, que chaque membre installera dans son propre compte.

   d. Une application Streamlit in Snowflake, exécutée sur le calcul du compte et soumise à sa gouvernance.

7. Votre entreprise porte 40 traitements Spark et veut fermer le cluster qui les fait tourner, sans tout réécrire.

   a. Snowpark Connect for Spark : le code Spark existant s'exécute sur le moteur Snowflake, sans cluster à gérer.

   b. Snowpark Container Services, qui relance le cluster Spark à l'intérieur de Snowflake.

   c. Rien à faire : il faut réécrire les 40 traitements avec l'API DataFrame de Snowpark.

© 2026 iAlexMG. Tous droits réservés.

d. Le connecteur Spark, qui déplace le calcul du cluster vers l'entrepôt sans toucher au code.

8. Vous avez bâti une logique de scorage et son écran, et vous voulez la livrer à des clients sans que leurs données sortent de leur compte.
    a. Un partage de données, qui leur donne accès à vos tables en lecture.
    b. Un reader account par client, que vous créez et payez.
    c. Le Native App Framework, qui empaquette logique et données en une application installée chez le consommateur.
    d. Une UDF que vous publiez et que chaque client appelle depuis son compte.
9. L'entraînement d'un modèle échoue faute de mémoire sur un entrepôt standard de taille LARGE. Le jeu de données tient pourtant sur le disque.
    a. Passer l'entrepôt en multi-cluster : plus de clusters, donc plus de mémoire.
    b. Passer sur un entrepôt Snowpark-optimized, qui offre plus de mémoire par nœud.
    c. Activer le Query Acceleration Service, qui prête de la mémoire serverless.
    d. Découper l'entraînement en UDF, exécutées ligne par ligne.

© 2026 iAlexMG. Tous droits réservés.

Corrigé expliqué — Cours 02

À lire après avoir répondu. Chaque réponse nomme la section à relire : ce que vous ratez ici vous dit quoi reprendre.

1. Réponse c. C'est l'idée de tout le module : on amène le calcul à la donnée. Snowpark pousse les opérations en SQL sur l'entrepôt (*pushdown*), et rien ne se télécharge, contre l'option a. L'option b décrit ce que Snowpark vous évite d'écrire. L'option d confond Snowpark, l'API native, avec Snowpark Connect, qui est le pont pour du code Spark (sections 1 et 2).

2. Réponse a. Un DataFrame Snowpark est paresseux : les opérations construisent un plan, et rien ne part tant qu'une action ne le déclenche. Un entrepôt suspendu se réveille seul quand une requête arrive, ce qui écarte l'option b ; Snowpark n'attend pas de `COMMIT` pour lire, contre l'option c ; et une erreur de filtre se lèverait à l'action, pas en silence, contre l'option d (section 2).

3. Réponse d. Une UDF calcule une valeur par ligne et s'utilise dans une requête ; une procédure fait des actions, enchaîne du SQL avec des conditions, et se lance par `CALL`. Les options a et b appliquent un seul outil aux deux besoins. L'option c inverse les rôles : une UDTF rend des lignes, elle n'orchestre rien (section 3).

4. Réponse b. Rendre plusieurs lignes par entrée, c'est la définition d'une UDTF, et elle s'appelle dans un `FROM … TABLE(...)`. L'option a décrit justement ce qu'une UDF ne fait pas : elle rend une valeur par ligne. L'option c passe par une écriture, là où la question demande une fonction utilisable dans une requête. L'option d confond une vue avec une fonction (section 3).

5. Réponses b et c. Cortex a deux familles : les fonctions d'IA pour le texte, `AI_SUMMARIZE` ici, et les fonctions ML pour le tabulaire, `FORECAST` ici. Les deux s'appellent en SQL. L'option a détourne un LLM d'un travail de série temporelle, que `FORECAST` fait mieux et sans invention. L'option d confond un type d'entrepôt avec un service d'IA : les fonctions Cortex sont serverless (section 6).

6. Réponse d. Streamlit in Snowflake est fait pour bâtir et héberger une application de données Python dans la plateforme, sans serveur externe, avec les rôles et les politiques du compte. Le notebook de l'option a est un atelier, pas une application livrée. L'option b prête à Snowsight l'exclusivité de l'interactif. L'option c sort l'artillerie de la distribution entre comptes pour un écran interne (section 5).

7. Réponse a. Snowpark Connect for Spark fait exécuter du code Spark par le moteur Snowflake, via le protocole Spark Connect, sans cluster Spark à administrer. L'option b décrit des conteneurs, pas ce pont. L'option c est la solution que Snowpark Connect permet justement d'éviter. L'option d décrit le connecteur Spark, qui relie un cluster Spark à Snowflake mais ne supprime pas le cluster (section 2).

8. Réponse c. Une Native App empaquette la logique et les données, et s'installe chez le consommateur : les données de vos clients ne bougent pas de leur compte. Le partage de l'option a va dans l'autre sens, ce sont vos données qu'il expose. Le reader account de l'option b sert à servir un non-client de Snowflake. L'option d livre un morceau de logique, pas une application (section 7).

9. Réponse b. Un entrepôt Snowpark-optimized donne plus de mémoire par nœud, ce que demande un entraînement qui déborde. Le multi-cluster de l'option a ajoute des clusters pour la concurrence, sans rien changer à la mémoire d'une requête. Le Query Acceleration Service de l'option c prête du calcul pour des scans gros et variables, pas de la mémoire à un entraînement. L'option d transformerait un entraînement global en calculs indépendants par ligne, ce qui n'est pas la même opération (section 2).

© 2026 iAlexMG. Tous droits réservés.

Points clés à retenir

- Idée du module : on amène le code et l'IA à la donnée, pour la proximité, le zéro infra externe et la gouvernance héritée. Deux portes : Snowpark (code) et Cortex (IA prête à l'emploi).
- Snowpark = Python/Java/Scala dans Snowflake ; son DataFrame est paresseux et poussé en SQL (*pushdown*) sur l'entrepôt, contrairement à pandas (mémoire locale). Variantes : Snowpark-optimized (mémoire), Container Services (conteneurs/GPU), Snowpark ML, Snowpark Connect for Spark (faire tourner du Spark sur Snowflake).
- Code réutilisable : UDF (une valeur), UDTF (des lignes/une table), procédure (orchestration, `CALL`), en SQL, JavaScript, Python, Java ou Scala.
- Notebooks = cellules SQL/Python/Markdown dans Snowsight, sur du calcul Snowflake. Streamlit in Snowflake = applis de données Python hébergées dans Snowflake.
- Cortex : fonctions d'IA en SQL (`AI_COMPLETE`, `AI_SUMMARIZE`, `AI_TRANSLATE`, `AI_SENTIMENT`…, serverless ; il faut `USE AI FUNCTIONS` sur le compte et le rôle `SNOWFLAKE.CORTEX_USER` ou `AI_FUNCTIONS_USER`), et les anciens noms `SNOWFLAKE.CORTEX.*` à reconnaître ; fonctions ML en SQL (`FORECAST`, `ANOMALY_DETECTION`, `CLASSIFICATION`, `TOP_INSIGHTS`) ; plus Search (RAG), Analyst (langage naturel → SQL), Document AI.
- Native App Framework = empaqueter une app qui tourne chez le consommateur ; sa distribution est au Cours 06.

© 2026 iAlexMG. Tous droits réservés.